

A High-SNR SRAM-LUT-Based In-Memory-Computing Macro Supporting Energy-Efficient INT8 and FP8 Computation

Xinhang Zou¹, Student Member, IEEE, Yujin Wang², Student Member, IEEE,
Luchang Lei³, Graduate Student Member, IEEE, Wenbin Jia, Student Member, IEEE,
Cheng Peng, Graduate Student Member, IEEE, Yutong An, Xuanshen Shu⁴, Graduate Student Member, IEEE,
Huazhong Yang⁵, Fellow, IEEE, Yongpan Liu⁶, Senior Member, IEEE, and Hongyang Jia¹, Member, IEEE

Abstract—Recent quantized neural networks require various dynamic ranges and resolutions to enable better inference accuracy, necessitating mixed support of energy-efficient integer and floating point computation. This work presents an SRAM look-up table (LUT) based INT8/FP8 dual mode in-memory-computing (IMC) macro which supports configurable FP8 (E4M3 and E5M2) multiply-accumulate (MAC) operation and 2:8 structural sparsity pruning. Instead of suffering either excessive overheads of shifting and alignment logic or accuracy degradation for extreme energy efficiency, this macro exploits fine-grain exponent sharing and efficient LUT-based MAC circuits to balance the two aspects. The key features are: 1) a pipelined and SNR-oriented mantissa and exponent execution circuit for FP8 multiplication; 2) a LUT-based shifter and 1-stage adder to reduce digital logic overhead in FP MAC; 3) an in-macro decoder to support 2:8 structural sparsity pruning. The silicon prototype in 28nm CMOS achieves 81.69% accuracy for DeiT-base model for ImageNet dataset in dense FP8 mode, and 76.15% accuracy for ResNet-34 model in CIFAR100 dataset in sparse FP8 mode. The 2:8 pruning provides a 31.42 TFLOPS/W peak energy efficiency and 3.56 TFLOPS/mm² peak area efficiency in sparse FP8 mode.

Index Terms—In-memory computing, deep learning, look-up table, floating-point computation.

I. INTRODUCTION

WITH the rapid development of AI task complexity, models, e.g., vision transformer (ViT), demand higher computational accuracy and energy efficiency. 8-bit floating-point formats (FP8) exhibit high training and inference accuracy at a low bit width [1]. In-memory-computing (IMC), as a promising alternative to conventional von Neumann architecture, featuring high compute parallelism and data locality, has shown exceeding integer (INT) multiply-accumulate (MAC) performance. However, the shifting and alignment in the original FP8 formats introduce large overheads, which are

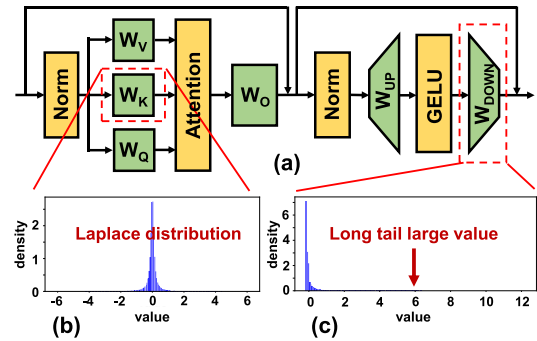


Fig. 1. (a) A typical ViT dataflow; (b) input data distribution in W_k layer; (c) input data distribution in W_{DOWN} layer.

unfriendly to IMC circuits. More importantly, the microarchitectural design impacts the computational noise significantly.

To avoid the shifting and alignment overhead, some floating-point-IMC works trimmed alignment steps assuming specific input activation (IA) and weight distributions, e.g., Gaussian, using coarse-grain pre-alignment and/or block-shared exponents. However, complex task scenarios, e.g., ViT shown in Fig. 1, employ irregular data distributions. Previous energy-efficient floating-point IMC designs might suffer accuracy loss due to alignment truncation, where signal-to-noise ratio (SNR) should be carefully characterized when trading off task accuracy and efficiency. To meet the needs for computation accuracy, recent digital floating-point IMC works adopted high-precision alignment and accumulation logic, but introduced huge energy and area costs, limiting gains on area efficiency, storage density, and energy efficiency. In conventional floating-point IMC works, a large portion of the area is attributed to digital logic for shifting and addition, as shown in Fig. 2 (a). Recent digital-IMC works [2], [3] adopted LUT to solve this resource mismatch issue in INT8 MAC operation, but LUT method for floating-point operation has not yet been explored.

In addition to trading off an appropriate SNR of MACs, another effective way to enhance the computation efficiency is to support sparsity. Previous algorithmic works [4], [5]

Received 3 March 2026; revised 23 April 2026; accepted 11 May 2026.
Date of publication 15 May 2026; date of current version 1 July 2026.
This work was supported in part by the Carbon Neutrality and Energy System Transformation (CNEST) Program and in part by NSFC under Grant 62374101. This brief was recommended by Associate Editor C. Meinhardt.
(Corresponding author: Hongyang Jia.)

The authors are with the Department of Electronic Engineering, Tsinghua University, Beijing 100084, China (e-mail: hjia@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TCSII.2026.3693836

1558-3791 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

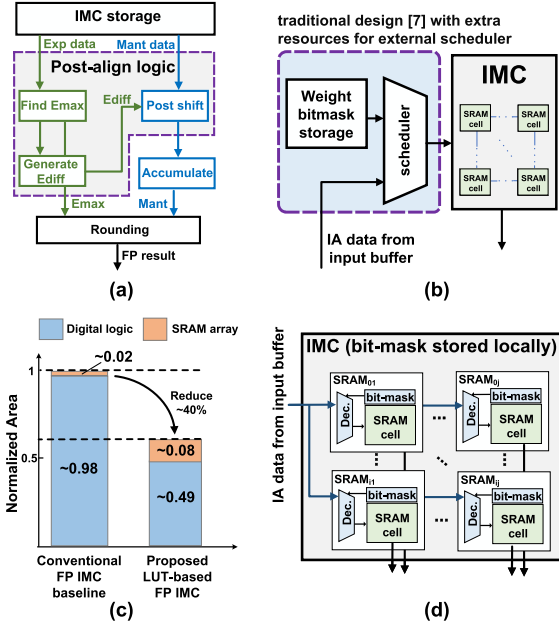


Fig. 2. (a) The additional circuit overhead required for the post-alignment process; (b) external schedulers introduce extra overhead in previous works [7]; (c) the area breakdown of a conventional FP IMC macro baseline and this work. The normalized area breakdown is derived from work [10]; (d) proposed in-macro decoders combined with local storage resources.

have demonstrated that structured sparsity can significantly enhance computational efficiency while ensuring inference accuracy. However, sparsity scheduling introduces additional circuit overhead. As the high computing parallelism of IMC is suitable for processing dense data, recent work [7] uses external out-macro schedulers to rearrange the input activation, shown in Fig. 2 (b), which takes about 15% area overhead.

To address the aforementioned challenges, this work employs an SNR-oriented design approach for digital floating-point IMC. Based on the SNR evaluation, this work implements a lookup-table (LUT) based SRAM IMC macro supporting both INT8 and FP8 computation, where pre-aligned and the first-stage-adder-tree results are pre-computed and stored in SRAM array. The LUT method save $\sim 40\%$ area overhead compared to conventional FP IMC baseline. The IMC macro supports both E4M3 and E5M2 FP8 formats. Moreover, it embeds sparse bit-mask information in the idle rows of LUT, as shown in Fig. 2 (d), enabling 2:8 pruning to further improve energy and area efficiency, while keeping ample accuracy for vision tasks. The key contributions of this work are listed as follows:

- We characterized how microarchitectural choices, e.g., rounding bit-width, pre-alignment grouping, affect the computational SNR of FP8 MAC operation and accuracy in vision tasks, guiding an implementation for a more balanced tradeoff between precision and efficiency.
- We fused the support for INT8/FP8 MAC operation in a compact unified LUT-based IMC circuits design.
- The proposed LUT-based INT8/FP8 dual-mode IMC macro also supports 2:8 sparsity in weights, achieving a peak 31.42 TFLOPS/W energy efficiency and 3.56 TFLOPS/mm² area efficiency in sparse FP8 mode.

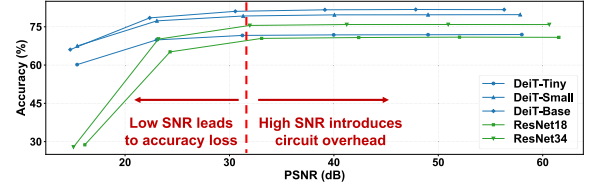


Fig. 3. The task-accuracy-versus-SNR tradeoff for FP8 computation.

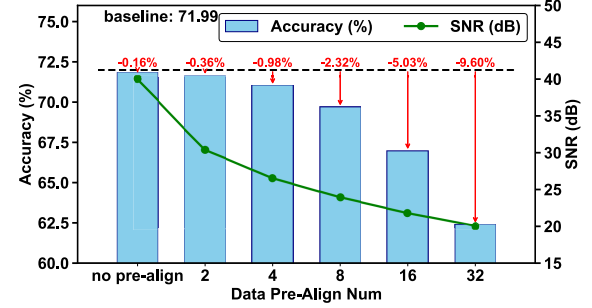


Fig. 4. The variation of task accuracy and SNR with pre-align granularity.

The demonstrated silicon prototype in 28 nm CMOS also shows 81.69% accuracy for DeiT-base model for ImageNet dataset in dense FP8 mode.

II. SNR CHARACTERIZATION OF FLOATING-POINT IMC

Similar to the SNR consideration in analog IMC [12], the truncation and rounding in floating-point alignment and accumulation potentially introduce computation noise. To better illustrate the relationship between the SNR of FP8 MAC and the task accuracy, which provides guidance to the IMC circuit design, Fig. 3 shows a series of simulations, where shift truncation is inserted in FP8 MAC operators. When the SNR of FP8 MAC reaches around 30dB, the inference accuracies of ViT and CNN models are close enough to those with standard full-precision FP8. When the SNR is less than 30dB, rounding noise dominates the accuracy degradation. These indicate that 30dB SNR is a sweet point for ViT-inference tasks balancing accuracy and energy efficiency, which is selected as the design target of this work.

The preserved bit-width of alignment is the main factor affecting the noise level of FP8 calculation. Keeping a 12b shifting space, this operator achieves a 40dB SNR compared with standard FP8 computation, leaving a margin for exploring pre-alignment. Fig. 4 explores the relationship between input pre-alignment and computation noise, where a pre-alignment of 2 inputs keeps the SNR at a 30dB level, guiding the design of this digital LUT-based IMC macro.

III. MACRO OVERVIEW

According to the aforementioned SNR consideration, this work proposes a digital FP IMC macro featuring: (1) a pipelined and SNR-aware mantissa and exponent execution circuit for multiplication; (2) a LUT-based shifter & 1-stage adder to reduce digital logic overhead in FP8 MAC; (3) an in-macro decoder for embedded 2:8 structural sparsity pruning.

Fig. 5 illustrates the overall architecture. The SRAM LUT-based INT8/FP8 dual-mode IMC macro is organized in

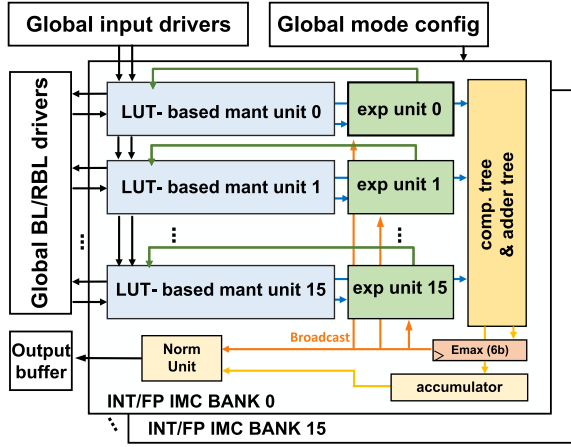


Fig. 5. Overall architecture of the INT8-FP8 dual-mode IMC macro.

16 banks which share input activations. Each bank has 16 SRAM-LUT mantissa units with 160b capacity, supporting MAC operation of 2 FP8 or 4 INT8 weights. In FP8 mode, each SRAM-LUT mantissa unit stores the possible results of mantissa alignment and addition in each pair of weights. Therefore, the first stage of mantissa alignment and addition is translated into the indexing of pre-computed results stored in the LUT, thus saving about 40% of the area cost while ensuring the computation accuracy, as shown in Fig. 2 (c). At the same time, exponent units pipeline with mantissa units to perform addition and comparison of input activation and weight exponent, generating the maximum of exponents and shifting length information for mantissa alignment. The post-align mantissas are accumulated and finally normalized to FP16 to complete FP8 MAC operation. In INT8 mode, each SRAM-LUT mantissa unit stores the possible results of 4 weights addition, and these exponent units are gated to reduce energy consumption. These form a 32-input FP8 or 64-input MAC operation in each bank of the macro. In order to explore sparsity for better computation efficiency in certain tasks, this work embeds the sparse scheduler in the decoder of each mantissa unit for logic reuse and tighter design, as shown in Fig. 2 (d). When the sparse mode is enabled, the decoder in the mantissa unit can filter the input data according to the 2:8 sparsity bit-mask stored in the LUT, thus supporting 32-input FP8 or 64-input MAC operation in each bank.

IV. CIRCUIT DESIGN FOR LUT-BASED COMPUTATION

A. Mantissa and Exponent Execution Pipeline

To execute post-align FP8 MAC, the differences between exponents and their global maximum need to be calculated in advance to control the shift alignment and accumulation of the mantissa. Each exp unit first computes the exponential sum of the two inputs and weights stored in a single LUT, then compares the two exponent sums and sends the maximum one to the comparator tree for calculating the global exponent maximum. This global maximum is broadcast to all exp units to calculate the shifting length of each post-alignment. After obtaining the shifting information, the mantissa of the input activation enters bit-serially into the macro. The sparse decoding with bit-mask and input-to-WL decoding are

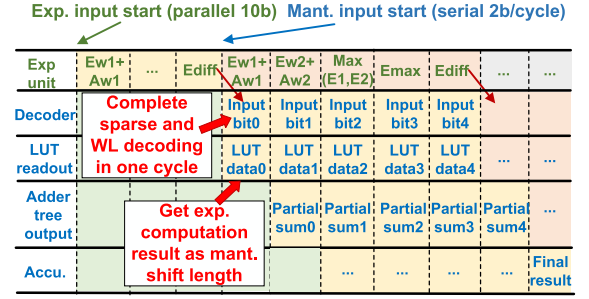


Fig. 6. Proposed mantissa and exponent execution pipeline.

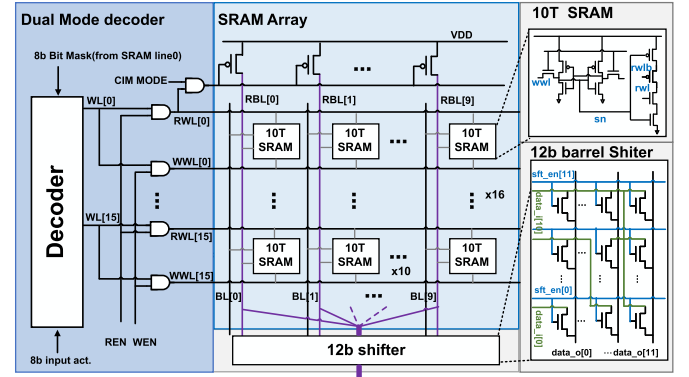


Fig. 7. Micro architecture design of LUT-based mantissa unit.

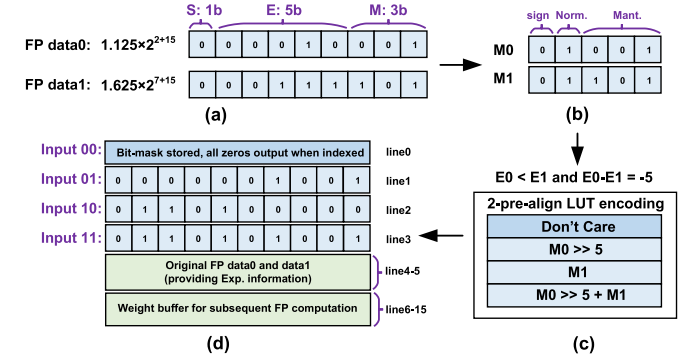


Fig. 8. The proposed LUT encoding and indexing method during pre-compilation. (a) Original weight data in FP8 format; (b) mantissa data with sign bit and normalization bit extension; (c) compare exponent data, then pre-align mantissa data to the exponent difference; (d) final encoded LUT storage.

completed in the same cycle. The desired sum result is selected from the LUT and post-aligned by the shifter, and then fed into the adder tree to obtain the partial-sum of a single-bit input. Since mantissa is extended to 5 bits in E5M3 format, it takes 5 cycles to accumulate and get the final mantissa result. For E4M3 format, the logic associated to the redundant exponent bit is gated, while in E5M2 format, the constant zero in mantissa leads to higher input sparsity reducing energy overhead. Finally, the mantissa result and exponent maximum are rounded and normalized to obtain the final calculation result in FP16 format. Fig. 6 shows the pipeline and the specific stages for calculating exponents and mantissas.

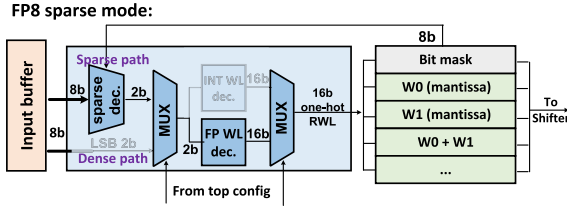


Fig. 9. In-macro indexing circuit of sparse FP8 mode.

B. LUT-Based Shifter & 1-Stage Adder

Fig. 7 represents the circuit-level design of the proposed LUT-based shifter & 1-stage adder, which mainly consists of SRAM storage, a WL decoder, and a shifter, while Fig. 8 shows detailed LUT encoding and indexing methods. Line 0 contains bit-mask information. The LUT is 10-bit wide, supporting shifting for 2-pre-aligned shift-and-add operation of the two 5 bits mantissas in each pair. Lines 1-3 take possible shift addition results for mantissas, while lines 4-5 contain the original weights under computation, and the remaining 10 lines contain weights for subsequent calculations. In INT8 mode, the LUT indexing shares a similar method with [2].

Fig. 7 also shows the detailed structure of a 10T SRAM and a 12-bit barrel shifter. The 10T SRAM consists of a standard 6T static storage structure and a 4T C2MOS inverting readout port, which improves the readout stability and reduces toggle rate while introducing negligible overhead [7]. In sparse IMC mode, since the bit-mask occupies line 0, a PMOS will drive the read bit-line (RBL) to VDD for an all-zero input to get a '0' readout. In SRAM mode, the 10T SRAM array can be read and written as standard memory.

C. In-Macro Index Dual Mode Decoder

Besides the benefits of converting MAC into indexing, sparse rearrangements of input data can be combined with row selection in the LUT. In dense computing mode, the decoder selects the pre-computed result stored in the LUT according to the input information. In INT8 mode, the WL decoder functions as a 4:16 one-hot decoder; while in FP8 mode, the WL decoder evaluates the 2-bit mantissa input to index the LUT. While in sparse mode, the sparse decoder first filters the input according to the bit-mask of the weight, and then send the filtered input to the WL decoder, making use of the sparse feature to improve efficiency. Moreover, sparse information, e.g., bit-mask, can be stored in line 0 of LUT, without the need for additional storage resources. Fig. 9 shows the dataflow of the sparse FP8 mode. The stored 2:8 structural sparsity bit-mask guides the decoder to filter the input data, where 2 of the 8 bit-serial inputs corresponding to non-zero weights are selected, thus skipping the MAC operations.

V. EXPERIMENTAL RESULTS

A. Chip Measurements

Fig. 10 shows the silicon prototype fabricated in 28nm CMOS technology. The macro occupies a 0.172mm² area with a total capacity of 40kb, working at 0.63-1.20V 50-750MHz for FP8 mode and 0.56-1.20V 50-1100MHz for INT8 mode at room temperature. It achieves a normal energy efficiency of 1.87 TFLOPS/W for FP8 and 2.32 TOPS/W for INT8 in

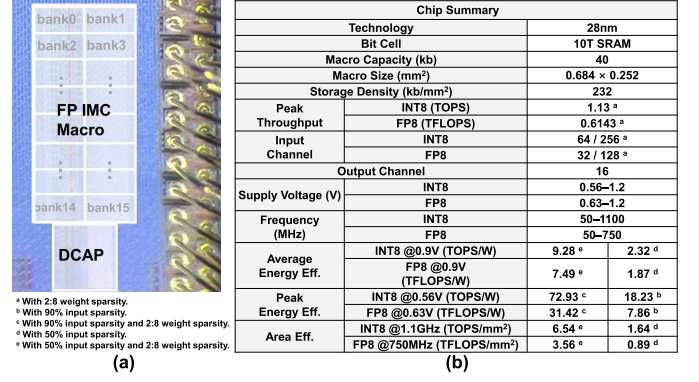


Fig. 10. (a) Die photo; (b) chip summary.

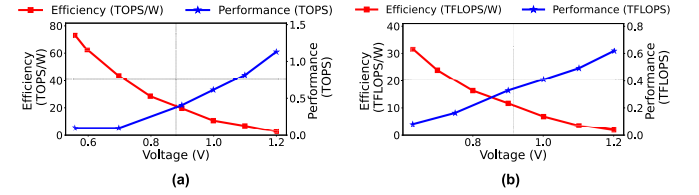


Fig. 11. (a) Energy-efficiency and throughput of INT8 mode; (b) FP8 mode, both measured with 2:8 structural weight sparsity and 90% input sparsity.

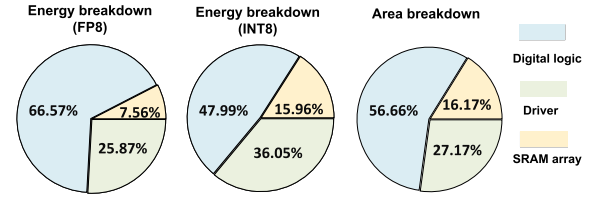


Fig. 12. Area and energy breakdown of the proposed dual-mode IMC macro. The energy of INT8/FP8 mode is measured with 2:8 structural weight sparsity and 90% input sparsity.

TABLE I
COMPARISON TABLE

work	ISSCC 2023[9]	ISSCC 2024[11]	ISSCC 2025[14]	ESSERC 2023[10]	ISSCC 2025[6]	ISSCC 2025[7]	This work
technology	28nm	16nm	28nm	28nm	28nm	28nm	28nm
Bit-cell	6T SRAM	4T eDRAM	SRAM	8/14T SRAM	9-10T SRAM	10T SRAM	10T SRAM
High SNR	No	No	No	Yes	Yes	Yes	Yes
Support precision	BF16/INT8	BF16/INT8	FP8/BF16/INT8/INT4	FP8/BF8	BF16/INT8	FP8/FP16/INT8	FP8/INT8
Frequency (MHz)	147	~250	~400	650/2400	~185	100-525	50-750
Supply Voltage (V)	0.6-0.9	0.4-0.8	0.55-0.9	0.55-1.2	0.55-0.9	0.62-0.9	0.63-1.2
Die Area (mm ²)	0.146	8	0.043	0.50	0.136	2.67	0.172
Storage density (kb/mm ²)	438	320	744	8	471	170	232
Peak throughput (TFLOPS)	0.3 ¹⁾ (BF16)	~0.80 ¹⁾ (BF16)	1 ¹⁾ (FP8)	0.668 (FP8)	0.0974 ³⁾ (BF16)	2.497 ⁴⁾ (FP8)	0.613 ²⁾ (FP8)
Peak energy efficiency (TFLOPS/W)	31.6 ¹⁾ (BF16)	91.2 ¹⁾ (BF16)	400 ¹⁾ (FP8)	12.1 (FP8)	62.84 ³⁾ (BF16)	31.2 ⁴⁾ (FP8)	31.42 ²⁾ (FP8)
Peak area efficiency (TFLOPS/mm ²)	2.05 ¹⁾ (BF16)	2.63 ¹⁾ (BF16)	2.34 ¹⁾ (FP8)	0.939 (FP8)	0.697 ³⁾ (BF16)	3.30 ⁴⁾ (FP8)	3.564 ²⁾ (FP8)

1) Using approximate computing, pre-alignment and/or coarse-grain block-shared exponents
 2) Using structural weight sparsity 3) Using adaptive-alignment and early stopping scheme
 4) Using structural weight sparsity and dynamic sparsity

dense mode with 50% IA sparsity and uniformly distributed weight, a peak energy efficiency of 31.42 TFLOPS/W for FP8 at 0.63V and 72.93 TOPS/W for INT8 at 0.56V in sparse

Dataset: ImageNet			
Model	Deit-tiny	Deit-small	Deit-base
Baseline [†]	71.99%	79.70%	81.76%
Top-1 Acc. [‡]	64.17%	75.24%	80.92%
Top-1 Acc. [‡]	71.63%	79.55%	81.69%
Acc. Loss [‡]	0.36%	0.15%	0.07%
SNR [‡]	30.39dB	32.70dB	34.85dB

[†]baseline is standard FP8 without 2:8 sparsity
[‡]evaluate with 2:8 sparsity and 2-pre-align

(a)

Dataset: CIFAR100			
Model	Resnet-18	Resnet-34	Resnet-50
Baseline [†]	76.22%	76.46%	77.46%
Baseline [†] *	75.97%	75.99%	77.43%
Top-1 Acc. [‡]	75.79%	76.15%	76.95%
Acc. Loss [‡]	0.43%	0.31%	0.51%
SNR [‡]	34.31dB	33.89dB	34.13dB

[†]baseline[†] is with 2:8 sparsity and after fine-tuning
[‡]evaluate with 2-pre-align and without 2:8 sparsity

(b)

Fig. 13. (a) SNR and accuracy loss of the proposed IMC macro in ViT models; (b) SNR and accuracy loss in CNN models with 2:8 pruning.

mode with 90% IA sparsity and 2:8 weight pruning. Fig. 11 shows the measured voltage scaling of the demonstrated macro with 2:8 structural sparsity enabled. Fig. 12 shows the area and power breakdown of the proposed IMC macro. Previous work [2] has claimed the importance of storage-computation area ratio consideration, since IMC also has the important function of serving as storage buffer. The LUT-based FP8 IMC macro achieves ~1:5 storage-computation area ratio even when including INT8 datapath. The use of LUT-based computation converts the MAC logic for both FP8 and INT8 into memory bitcells, alleviating the mismatch between storage and computation costs.

Table I compares to state-of-the-art works, showing exceeding performance and efficiency for FP8 inference. This work achieves higher calculation accuracy than the works exploiting data distribution for circuit simplification, and demonstrates higher area and energy efficiency than the works incurring additional circuit costs to achieve ideal precision. Thanks to the careful characterization of computation SNR and efficient LUT-based MAC design, this work shows balanced computation precision and efficiency.

B. Demonstrations

Fig. 13 shows the SNR and accuracy loss of the proposed IMC macro in different models, with all convolutional and fully connected layers mapped to the macro. In CNN models, the original dense FP16 weight data in linear layers are fine-tuned with 2:8 sparsity, rounded to FP8 format to complete the training steps. Then the FP8 sparse weight data are 2-pre-aligned in pairs and stored in the LUT to complete the inference evaluation steps. The accuracy loss introduced by 2:8 sparsity can be reduced and endurable after fine-tuning. The over 30dB SNR helps an average accuracy degradation below 0.4% of all demonstrated models. And the accuracy loss of 2:8 pruning is less than 0.51%. In ViT models, linear layers are replaced with the proposed 2-pre-align FP8 operator while keeping non-linear layers such as attention and normalization at full FP16 precision to evaluate the inference accuracy. The last classification layer and dynamic matrix multiplication in multi-head attention are not mapped since these layers are usually mapped to conventional digital accelerators due to lower weight reuse. The Deit-base model shows 0.84% accuracy loss when applying 2:8 sparsity.

VI. CONCLUSION

This work exploits an SNR-oriented approach to design FP operators in IMC circuits. Based on the SNR evaluation, this work presents an SRAM-LUT-based INT8/FP8 dual-mode

IMC macro, featuring replacing shifters and adders computing with LUT indexing. It employs a high precision mantissa and exponent execution pipeline, a LUT-based shifter and 1-stage adder, and an in-macro decoder, achieving higher than 30dB SNR in dense FP8 mode, and 31.42 TOPS/W and 3.56 TOPS/mm² energy/area efficiency in sparse FP8 mode, with negligible accuracy loss in several vision tasks.

REFERENCES

- [1] (2022). *NVIDIA Hopper Architecture Whitepaper*. [Online]. Available: <https://www.advancedclustering.com/wp-content/uploads/2022/03/gtc22-whitepaper-hopper.pdf>
- [2] Y. He et al., “34.7 A 28 nm 2.4Mb/mm² 6.9–16.3TOPS/mm² eDRAM-LUT-based digital-computing-in-memory macro with in-memory encoding and refreshing,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 578–580.
- [3] C.-F. Lee et al., “A 12 nm 121-TOPS/W 41.6-TOPS/mm² all digital full precision SRAM-based compute-in-memory with configurable bit-width for AI edge applications,” in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Honolulu, HI, USA, Jun. 2022, pp. 24–25.
- [4] (2021). *NVIDIA Ampere Architecture Whitepaper*. [Online]. Available: <https://images.nvidia.cn/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>
- [5] A. Zhou et al., “Learning N:M fine-grained structured sparse neural networks from scratch,” in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–15. [Online]. Available: https://openreview.net/pdf?id=K9bw7vqp_s
- [6] X. Wang et al., “14.3 A 28 nm 17.83-to-62.84TFLOPS/W broadcast-alignment floating-point CIM macro with non-two’s-complement MAC for CNNs and transformers,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2025, pp. 254–256.
- [7] Z. Yue et al., “14.4 A 51.6TFLOPS/W full-datapath CIM macro approaching sparsity bound and <2–30 loss for compound AI,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2025, pp. 1–3.
- [8] S. Liu et al., “16.2 A 28 nm 53.8TOPS/W 8b sparse transformer accelerator with in-memory butterfly zero skipper for unstructured-pruned NN and CIM-based local-attention-reusable engine,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 250–252.
- [9] F. Tu et al., “A 28 nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 1–3.
- [10] J. Saikia, A. Sridharan, I. Yeo, S. Venkataramanaiah, D. Fan, and J.-S. Seo, “FP-IMC: A 28 nm all-digital configurable floating-point in-memory computing macro,” in *Proc. IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, Lisbon, Portugal, Sep. 2023, pp. 405–408.
- [11] W.-S. Khwa et al., “34.2 A 16 nm 96Kb integer/floating-point dual-mode-gain-cell-computing-in-memory macro achieving 73.3–163.3TOPS/W and 33.2–91.2TFLOPS/W for AI-edge devices,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 568–570.
- [12] H. Jia, H. Valavi, Y. Tang, J. Zhang, and N. Verma, “A programmable heterogeneous microprocessor based on bit-scalable in-memory computing,” *IEEE J. Solid-State Circuits*, vol. 55, no. 9, pp. 2609–2621, Sep. 2020.
- [13] A. Guo et al., “A 28 nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 128–130.
- [14] Y. Yuan et al., “14.5 A 28 nm 192.3TFLOPS/W accurate/approximate dual-mode-transpose digital 6T-SRAM CIM macro for floating-point edge training and inference,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2025, pp. 258–260.
- [15] S. Um et al., “Dial: An energy-efficient dram in-memory computing accelerator with compact partial product lut and twisted differential adc,” in *Proc. Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Kyoto, Japan, Jun. 2025, pp. 1–3.